

Отчет по лабораторной работе №23 по курсу «Алгоритмы и структуры данных»

Студент группы М8О-116БВ-25 Сиухин Ярослав Алексеевич, № по списку 22

Контакты www, e-mail: zaq2007q@gmail.com

Работа выполнена: «03» июня 2026 г.

Преподаватель: Речинская Ангелина Юрьевна

Отчет сдан «__» _____ 2026 г., итоговая оценка _____

Подпись преподавателя _____

Тема	Бинарное дерево поиска и операции над деревом.
Цель	Разработать программу для создания, изменения, визуализации и анализа бинарного дерева поиска.
Задание	Реализовать вставку и удаление узлов дерева, вывод дерева на экран и поиск значения нетерминальной вершины, расположенной на максимальной глубине.
Файлы	/Users/siuxin_yaroslav/Desktop/projectsLabsC/second_semestr/lr23

Оборудование и программное обеспечение

- Компьютер: Apple Silicon, архитектура arm64.
- Операционная система: macOS 15.7.3.
- Интерпретатор команд: bash 3.2.57.
- Система программирования: Apple clang 17.0.0, стандарт C11.
- Редактор текстов: nano / среда разработки и терминал macOS.

Идея, метод и алгоритм

Дерево строится по правилу бинарного дерева поиска: меньшие значения помещаются в левое поддереву, остальные - в правое. Для удаления узла с двумя потомками используется минимальный узел правого поддерева.

1. Инициализировать пустой корень дерева.
2. При вставке рекурсивно спускаться в левое или правое поддерево до свободной позиции.
3. При удалении обработать случаи листа, одного потомка и двух потомков.
4. Для визуализации выполнить рекурсивный обход с увеличением отступа на каждом уровне.
5. Для поиска требуемой вершины рекурсивно обойти дерево и запомнить нетерминальную вершину с максимальной глубиной.
6. Перед выходом освободить память всех узлов дерева.

Сценарий выполненной работы

Файл: lab23.c

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *left;
    struct Node *right;
} Node;

Node* insert(Node *root, int value) {
    if (root == NULL) {
        Node *newNode = (Node*)malloc(sizeof(Node));
        newNode->data = value;
        newNode->left = newNode->right = NULL;
        return newNode;
    }
    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);
    return root;
}

Node* findMin(Node *root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

Node* deleteNode(Node *root, int value) {
    if (root == NULL) return NULL;
    if (value < root->data)
        root->left = deleteNode(root->left, value);
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
    else {
        if (root->left == NULL) {
            Node *temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            Node *temp = root->left;
            free(root);
            return temp;
        }
        Node *temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void printTree(const Node *root, int depth) {
    if (root == NULL) return;
    for (int i = 0; i < depth; i++) printf("  ");
    printf("%d\n", root->data);
    printTree(root->left, depth + 1);
    printTree(root->right, depth + 1);
}

void findMaxDepthNonTerminal(const Node *root, int depth, int *maxDepth, int *resultValue) {
    if (root == NULL) return;
    if (root->left != NULL || root->right != NULL) {
        if (depth > *maxDepth) {
            *maxDepth = depth;
            *resultValue = root->data;
        }
    }
    findMaxDepthNonTerminal(root->left, depth + 1, maxDepth, resultValue);
    findMaxDepthNonTerminal(root->right, depth + 1, maxDepth, resultValue);
}
```

```

int getMaxDepthNonTerminalValue(const Node *root) {
    int maxDepth = -1;
    int result = 0;
    findMaxDepthNonTerminal(root, 0, &maxDepth, &result);
    if (maxDepth == -1) {
        printf("Дерево пусто или не содержит нетерминальных вершин.\n");
        return 0;
    }
    return result;
}

void freeTree(Node *root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    Node *root = NULL;
    int choice, value;

    do {
        printf("\nМеню:\n");
        printf("1. Добавить узел\n");
        printf("2. Визуализировать дерево\n");
        printf("3. Удалить узел\n");
        printf("4. Найти значение нетерминальной вершины с максимальной глубиной\n");
        printf("5. Выход\n");
        printf("Ваш выбор: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Введите целое число: ");
                scanf("%d", &value);
                root = insert(root, value);
                printf("Узел добавлен.\n");
                break;
            case 2:
                if (root == NULL) {
                    printf("Дерево пусто.\n");
                } else {
                    printf("Текущее дерево:\n");
                    printTree(root, 0);
                }
                break;
            case 3:
                if (root == NULL) {
                    printf("Дерево пусто.\n");
                } else {
                    printf("Введите значение для удаления: ");
                    scanf("%d", &value);
                    root = deleteNode(root, value);
                    printf("Узел удалён (если существовал).\n");
                }
                break;
            case 4:
                if (root == NULL) {
                    printf("Дерево пусто.\n");
                } else {
                    int res = getMaxDepthNonTerminalValue(root);
                    printf("Значение нетерминальной вершины с максимальной глубиной: %d\n", res);
                }
                break;
            case 5:
                printf("Выход из программы.\n");
                freeTree(root);
                break;
            default:
                printf("Неверный выбор. Попробуйте снова.\n");
        }
    } while (choice != 5);

    return 0;
}

```

Распечатка протокола

```
$ gcc -Wall -Wextra -std=c11 -o lr23 lab23.c
$ ./lr23
```

Добавлены узлы: 10, 5, 15, 3, 7

Текущее дерево:

10

5

3

7

15

Значение нетерминальной вершины с максимальной глубиной: 5

Удаление узла 5

Текущее дерево:

10

7

3

15

Дневник отладки

№	Дата	Событие	Действие по исправлению	Примечание
1	03.06.2026	Проверка компиляции	Сборка выполнена с ключами -Wall -Wextra -std=c11.	Критических ошибок не обнаружено.
2	03.06.2026	Проверка тестового сценария	Запущен контрольный ввод, результат сопоставлен с ожидаемым поведением.	Программа работает корректно.
3	03.06.2026	Контроль памяти	В коде предусмотрено освобождение динамических структур, где они используются.	Замечаний по отчету нет.

Замечания автора по существу работы

Программа реализована в соответствии с заданием. Проверка выполнялась на контрольных данных, приведенных в протоколе.

Выводы

В ходе выполнения работы я изучил основные операции над бинарным деревом поиска, рекурсивные обходы и обработку разных случаев удаления узла.

Недочеты при выполнении задания могут быть устранены путем расширения набора тестов и добавления дополнительной проверки некорректного пользовательского ввода.

Подпись студента _____